

Problem 1

Meredith has just bought a new car and needs license plates. In Pennsylvania, license plates consist of three uppercase letters followed by four digits. License plates are given out in *lexicographic order*, meaning that plate AAA 0000 is first, followed by AAA 0001, AAA 0002, etc.,. After AAA 9999 comes plates AAB 0000, AAB 0001, etc., and after plate AZZ 9999 comes plate BAA 0000. The very last plate is ZZZ 9999.

For most people, any old license plate will do, but not for Meredith. She insists that the 4 digit number on the plate must be a prime number. Recall that a prime number is a number > 1 whose only factors are 1 and the number itself. The first few prime numbers are

2, 3, 5, 7, 11, 13, 17, 19, 23, . . .

So if Meredith goes to the DMV to get a set of plates, and the next available plate is JPB 1285, she'll wait until plate JPB 1289 becomes available, since 1289 is the first prime greater than or equal to 1285. If the next available plate had been JPB 9999, she would have waited for the plate JPC 0002.

Perhaps you see what we're driving at here. Starting at a given license plate, find the first plate lexicographically equal or greater that contains a prime number.

As a side note, this past January the largest prime number known to date was discovered. It is equal to $2^{74,207,281} - 1$ and contains 22,338,618 digits. A little too large to fit on a license plate.

Input

The input file contains multiple test cases. Each test case is on a single line and contains a three character string followed by a 4 digit number. The three character string consists solely of uppercase letters other than the string "ZZZ". A line containing END 0000 terminates input.

Output

For each test case, display the license plate Meredith will accept.

Sample Input 1

```
JPB 1285
JPB 9999
END 0000
```

Sample Output 1

```
JPB 1289
JPC 0002
```

Problem 2

Do you know how long one million seconds is? How about one billion seconds? Well, the answer to the first one is

11 days, 13 hours, 46 minutes and 40 seconds

Accurate, but a little over-precise. A more reasonable answer might be “11 days, 14 hours” or just “12 days” (after rounding). As another example, if I asked how long three million inches were, you could approximate it by saying either “47 miles” or “47 miles, 613 yards”. This sort of imprecision is what we’re looking for in this problem!

Input

Input consists of two lines. The first line has the form

$n \text{ } name_1 \text{ } c_1 \text{ } name_2 \text{ } c_2 \dots name_{n-1} \text{ } c_{n-1} \text{ } name_n$

Each $name_i$ is a unit of measurement and each c_i is a conversion rate, telling you how many units of type $name_{i+1}$ are in a unit of type $name_i$. For example, in the first sample input, this line tells you that there are 24 hours in one day, 60 minutes in each hour and 60 seconds in each minute. The value n indicates the total number of unit names and satisfies $2 \leq n \leq 10$. All names are alphabetic strings with no blanks and always specify the plural form of the unit (which you should use for output no matter the value is associated with the unit). All conversion values are positive integers. Following this line is a single positive integer $m \leq 2,000,000,000$ indicating an amount of the last unit (i.e., $name_n$).

Output

Display two lines. The first line should give the closest conversion of the input amount to units specified by $name_1$, rounded to the nearest integer. The second line should do the same, but be a bit more precise using the two largest units – $name_1$ and $name_2$. When rounding, always round up when the fraction is ≥ 0.5 . Note that there are no commas in the output.

Sample Input 1

```
4 days 24 hours 60 minutes 60 seconds
1000000
```

Sample Output 1

```
12 days
11 days 14 hours
```

Sample Input 2

```
5 years 365 days 24 hours 60 minutes 60 seconds
1000000000
```

Sample Output 2

```
32 years
31 years 259 days
```

Sample Input 3

```
2 feet 12 inches
11
```

Sample Output 3

```
1 feet
0 feet 11 inches
```

Sample Input 4

```
4 jiffys 2 lambtailshakes 11 flashes 3 twinklings
20902
```

Sample Output 4

```
317 jiffys
316 jiffys 1 lambtailshakes
```

Problem 3

Peter is expecting a large shipment of fuel oil, but he has a small problem (doesn't everyone in these programming problems!). The only containers he has are a set of large bottles (each with the same volume) and a set of smaller bottles (also each with the same, but smaller volume). Given the volume of the shipment of oil, he would like to store the oil in the bottles so that

1. all of the oil is stored,
2. each bottle is filled to the top, and
3. the minimum number of bottles is used.

While Peter thinks he has solved this problem for his given bottle sizes, he often spends hours wondering what would happen if his bottles had different volumes (apparently Peter doesn't lead the most exciting life).

Input

The input consists of a single line containing three positive integers s v_1 v_2 , where $s \leq 10^6$ is the volume of the shipment, and $v_1, v_2 \leq 10^6$ are the volumes of the two types of bottles, with $v_1 > v_2$.

Output

Output the number of bottles of size v_1 and the number of bottles of size v_2 which satisfy Peter's two conditions. If the conditions cannot be met, output `Impossible`.

Sample Input 1

1000 9 7

Sample Output 1

108 4

Sample Input 2

1000 900 7

Sample Output 2

Impossible

Sample Input 3

1000 10 7

Sample Output 3

100 0

Problem 4

Mastermind is a two-person code breaking game which works as follows. The first person (the *code maker*) creates a sequence of n colored pegs (with duplicate colors allowed) and hides it from view. This sequence of pegs is the *code*.

The second person (the *code breaker*) has the job of trying to determine the code maker's code and she does so by making a series of guesses. Each guess also consists of n colored pegs. After each guess, the code maker gives the code breaker feedback about how close she is. This feedback consists of two number r and s , where

- r = the number of pegs that are identical in both color and position in the code and the guess, and
- s = the number of remaining pegs that are identical in color but not in the same position.

For example, if the code is BACC (where we use different letters to represent colors) and the guess is CABB, then $r = 1$ (the A in the second position of both the code and the guess) and $s = 2$ (a B and C in the remaining three characters). Note that only one of the B's in the guess will “match” with the single B in the code: once pegs in the code and the guess have been “matched” with each other, they can't be matched with any other pegs.

Your job in this problem is to determine r and s given a code and a guess.

Input

The input is a single line containing a positive integer $n \leq 50$ (the length of the code) followed by two strings of length n — the first of these is the code and the second is the guess. Both code and guess are made up of upper-case alphabetic characters.

Output

Output the values of r and s for the given input.

Sample Input 1

4 BACC CABB

Sample Output 1

1 2

Sample Input 2

13 ABCDEFGHIJKLM NOPQRSTUVWXYZ

Sample Output 2

0 0
